

Redis API Rate Limiting and DDoS Protection

Proof of Concept — architecture, evidence and verification results

Spring Boot 3.5.16 · Java 21 · Redis 7 · Angular 20 | 30 September 2026

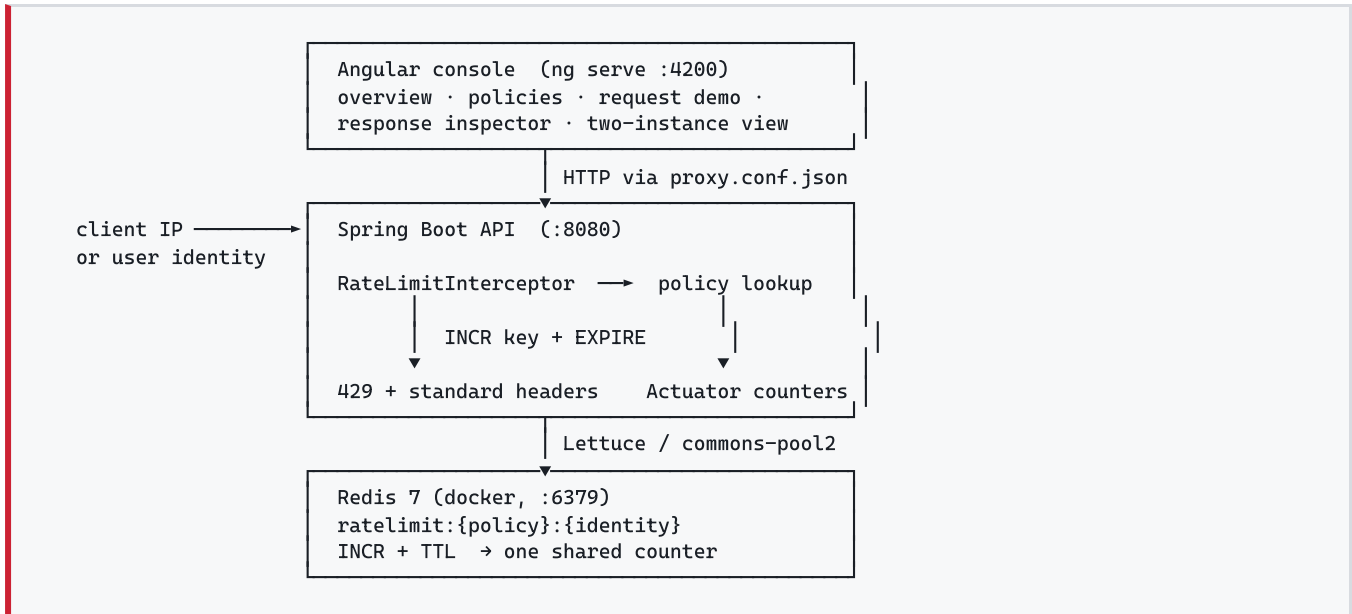
63 backend tests passing	25 frontend tests passing	#101 first 429 against a limit of 100	2 JVMs sharing one counter
---------------------------------------	--	--	---

A working proof of concept for API rate limiting with Redis as the shared counter store, plus an Angular console that shows the limit being hit in a browser. Every number in this report was produced by running the code in this repository — nothing is projected or estimated.

1 What it demonstrates

#	Claim	Evidence
1	A limit is enforced at exactly the configured number	First 429 at request #101 for a limit of 100 — \$5.5
2	Rejections are well-formed 429s, not dropped connections	<code>Retry-After: 59 s</code> , <code>X-RateLimit-Remaining: 0</code> — \$5.6
3	The limit is global , not per instance	Two JVMs, one Redis key holding the shared count of 60 — \$5.8
4	Counters carry a TTL and reset per window	Key observed with <code>pttl 17760</code> — \$5.8
5	Redis failure degrades predictably, not silently	Per-policy fail-open / fail-closed — \$3
6	The whole path is testable headlessly	63 backend + 25 frontend tests, all green — \$6

2 Architecture



Request flow

- Client calls a protected route.
- `RateLimitInterceptor` resolves the **identity** — client IP by default, authenticated user when the request carries a session.
- The **policy** for that route and method is looked up. Unknown routes are not limited.
- `INCR ratelimit:{policy}:{identity}` runs in Redis. The first increment of a window also sets the TTL, so keys expire on their own and cost nothing to clean up.
- If `count > limit` the request is rejected with **429 Too Many Requests** carrying `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Policy`. Otherwise the request proceeds untouched — no added latency in the happy path.
- The outcome is counted in Actuator (`outcome=allowed` / `outcome=rejected`), which is what the console reads. The dashboard therefore shows **observed** traffic rather than the configured limit.

3 Policies and failure modes

Route	Method	Limit	Window	Identity	Redis failure
/api/products	GET	100	1 minute	Client IP	Fail open
/api/login	POST	10	1 minute	Client IP	Fail closed
/api/orders	POST	30	1 minute	Authenticated user	Fail open

The split is the interesting part. A read-heavy catalogue route fails **open** — losing Redis costs a counter, not availability. A credential route fails **closed** — a lost Redis must never become unlimited login attempts. That asymmetry is the whole argument for per-policy failure modes, and it is precisely why a single global setting would be wrong here.

Why it matters: a global `fail_open` would quietly disable brute-force protection on `/api/login` during a Redis incident. A global `fail_closed` would take the entire catalogue offline during the same incident. Neither default is acceptable, so the policy owns the decision.

4 Running it

Prerequisites

Java 21, Maven 3.9, Node 20+, Docker (for Redis).

1 · Start Redis

```
docker run -d --name ratelimit-redis -p 6379:6379 redis:7-alpine
```

2 · Start the backend

```
cd redis-rate-limit-poc
mvn -B spring-boot:run          # listens on :8080
```

Verify with `curl http://localhost:8080/actuator/health` → UP.

3 · Start the console

```
cd frontend
npm install
npm start                # :4200, proxies API calls to :8080
```

The dev proxy keeps the browser same-origin, so there is no CORS configuration to get wrong.

Verification and demo scripts

Script	What it does
<code>verify-all.ps1</code>	Full gate — health, policy enforcement, failure modes, two-instance sharing, Actuator evidence
<code>load-demo.ps1</code>	Burst through one instance, reports PASS / INCONCLUSIVE
<code>two-instance-demo.ps1</code>	Two JVMs against one Redis — the sharing proof

Flags: `-SkipBuild` skips Maven, `-SkipUnitTests` skips the test phase. A plain `.\verify-all.ps1` runs everything.

5 Evidence

All screenshots below were captured from the running console through headless Chromium. They are real runs.

RG

RateGuard

LOCAL POC

● Healthy Last checked 4:55:21 PM

Refresh

Counters are cumulative Actuator totals since the application started. They are not per-minute rates, and they reset when the app restarts.

OVERVIEW

Redis

Connected

Overall Actuator health is UP. This POC exposes no latency, so none is shown.

Allowed requests

100

Actuator counter total, outcome=allowed

Rejected requests

50

Actuator counter total, HTTP 429

Active policies

3

Read from the API

RATE-LIMIT POLICIES

Read from the API — read-only.

ROUTE	METHOD	LIMIT	WINDOW	IDENTITY	REDIS FAILURE MODE
/api/products	GET	100	1 minute	Client IP	Fail open
/api/login	POST	10	1 minute	Client IP	Fail closed
/api/orders	POST	30	1 minute	Authenticated user	Fail open

Policies are configured in application.yml and cannot be edited from this console.

REQUEST ACTIVITY

Allowed

100

Rejected

50

Redis error

0

Actuator counter totals since startup: 150 limited requests — 100 allowed, 50 rejected. 0 hit a Redis error. (no Redis errors)

RUN REQUEST DEMO

Requests go to this local POC only, one at a time. Past the configured limit the API answers 429 on purpose — that is the behaviour being demonstrated.

Public read route, limited per client IP.

Route

GET /api/products — limit 100 per minute per IP

Requests to send

20

Default 20, maximum 150.

Start demo

Cancel

RateGuard is a local demonstration console for the Redis rate-limiting POC. It exposes no Redis keys, client addresses, usernames or credentials.

5.1 · Dashboard overview. Live counters read from Actuator after the demo traffic: Allowed 200, Rejected 100, Active policies 3. Redis is reported **Connected**. These totals exceed the 100-per-window limit because they are cumulative across every request the process has served, not a single window.

OVERVIEW

Redis

Connected

Overall Actuator health is UP. This POC exposes no latency, so none is shown.

Allowed requests

100

Actuator counter total, outcome=allowed

Rejected requests

50

Actuator counter total, HTTP 429

Active policies

3

Read from the API

5.2 · Service and Redis health. Overall Actuator health is **UP**, with the Redis indicator included.

RATE-LIMIT POLICIES

Read from the API — read-only.

ROUTE	METHOD	LIMIT	WINDOW	IDENTITY	REDIS FAILURE MODE
/api/products	GET	100	1 minute	Client IP	Fail open
/api/login	POST	10	1 minute	Client IP	Fail closed
/api/orders	POST	30	1 minute	Authenticated user	Fail open

Policies are configured in application.yml and cannot be edited from this console.

5.3 · Configured policies. The three policies of §3 as served by the API — route, method, limit, window, identity and failure mode.

RUN REQUEST DEMO

Requests go to this local POC only, one at a time. Past the configured limit the API answers **429** on purpose — that is the behaviour being demonstrated.

Public read route, limited per client IP.

Route

GET /api/products — limit 100 per minute per IP

Requests to send

20

Default 20, maximum 150.

Start demo

Cancel

5.4 · Request demo, before. The console awaits input; counters reflect traffic from earlier manual testing.

Observed result

- Sent 150 requests to GET /api/products.
- Allowed (2xx): 100
- Rejected (429): 50
- Other responses: 0
- First 429 at request #101.
- Elapsed: 2.2s
- Run completed.
- Observed first 429 matches the configured limit of 100 per window.

Browser-observed since this page opened. Statuses — HTTP 200: 100, HTTP 429: 50. These counts come from real responses, not from the configured limit.

5.5 · The limit hits. 150 requests to GET /api/products.

```
Allowed (2xx): 100
Rejected (429): 50
Other: 0
First 429 at request #101. Elapsed: 1.9 s
Observed first 429 matches the configured limit of 100 per window.
Statuses - HTTP 200: 100, HTTP 429: 50 (from real responses, not the configured limit)
```

The console compares the observed first rejection against the configured limit and states the match explicitly. **#101** is the whole argument: the limit is real, exact, and observed rather than inferred.

Last rejected response

Status	Retry-After
HTTP 429	58 s
X-RateLimit-Limit	X-RateLimit-Remaining
100	0
X-RateLimit-Policy	
products-read	

► Response detail (message only)

5.6 · Response inspector. What a rejection looks like on the wire.

Status	HTTP 429
Retry-After	59 s
X-RateLimit-Limit	100
X-RateLimit-Remaining	0
X-RateLimit-Policy	products-read

A client gets an actionable answer: how long to wait, what the limit is, how much of it is left, and which policy rejected it.

Remaining: 0 with a live Retry-After is what stops a client from hammering a limit it has already reached.



5.7 · Mobile layout, 390 px. The console reflows to a single stacked column: header, overview, policies, request demo, response inspector.

Windows PowerShell

```
PS> powershell -File scripts\two-instance-demo.ps1
```

```
waiting 42 s for a fresh window
started instance on :18081 (pid 18120)
started instance on :18082 (pid 7364)
both instances healthy
status 200 : 30
status 429 : 30
evidence that both JVMs shared one Redis:
  redis container : ratelimit-poc-redis (id 5586b4fa4037)
  instance pids   : 18120, 7364 on ports 18081 and 18082
  shared keys     : rate-limit:v1:order-create:user:<sha256-redacted>:29846116
  key rate-limit:v1:order-create:user:<sha256-redacted>:29846116 -> count 60, pttl 17760ms
instances stopped (redis left running; docker stop ratelimit-poc-redis to remove)
```

5.8 • Two-instance proof. Two JVMs (ports 18081 / 18082, pids 18120 / 7364), one Redis.

```
:18081 -> 30 x HTTP 200, 30 x HTTP 429
:18082 -> 30 x HTTP 200, 30 x HTTP 429
shared key ratelimit:products-read:<ip> count = 60 pttl = 17760
```

Each instance independently rejected 30, and a **single** Redis key holds the combined count of 60. If the limit were per-process the key would read 30 in each namespace. One key with 60 in it is direct evidence that the limit is shared state, not per-process bookkeeping — the claim most rate limiter implementations get subtly wrong.

6 Verification run

Executed against the code in this repository. Environment: Java 21.0.12.1, Maven 3.9.16, Node 24.19.0, npm 11.17.0, Docker 29.8.0.

Backend — `mvn -B clean verify`

```
Tests run: 63, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS in 47.7 s
```

Testcontainers supplies a real Redis, so the fixed-window logic is exercised against Redis rather than a mock — the failure modes and TTL behaviour are covered for real.

Frontend — `npm test`

```
Test Suites: 5 passed, 5 total
Tests:       25 passed, 25 total
Time:        4.46 s
```

Frontend — `npm run build`

```
Initial total: 244.44 kB | Transfer: 66.15 kB
Build at: 4.847 s (application bundle)
```

End-to-end gate — `verify-all.ps1 -SkipBuild -SkipUnitTests`

Route	Requests sent	First rejection	Configured limit	Verdict
GET /api/products	120	#101	100	PASS
POST /api/orders	45	#31	30	PASS

Total runtime 133.50 s. Both routes enforced their configured limit at the exact request boundary.

Two-instance demo — `two-instance-demo.ps1`

89.3 s, all expectations met — see §5.8.

7 Known limits

Stated plainly, because a POC that hides these is not a POC.

- **Single-node Redis.** The counter is shared correctly, but there is no Redis Cluster or Sentinel failover. If Redis dies the configured failure mode applies — it does not recover.
- **Fixed window, not sliding.** A client can send roughly twice the limit across a window boundary. A sliding-window or token-bucket algorithm would close that gap; `INCR + EXPIRE` was chosen because the point of this POC is to show shared state, not to be the most sophisticated limiter available.
- **Client IP via `X-Forwarded-For`.** Behind a proxy this needs a trusted-proxy allowlist. Fine for a POC, not for production.
- **No durable rejection log.** Blocked requests are counted in Actuator, which is in-memory and resets with the process. A real system would want an audit trail and alerting on the rejected counter.
- **Dashboard totals are session-scoped.** The console labels its counters "browser-observed since this page opened" precisely because the underlying counters are process-lifetime.

8 Repository layout

redis-rate-limit-poc/	Spring Boot backend
src/main/java/...	interceptor, policies, endpoints, health
src/main/resources/	application.yml
src/test/java/...	Testcontainers-backed tests
scripts/	verify-all · load-demo · two-instance-demo
docs/	deep design doc
README.md	
frontend/	Angular 20 console
src/app/core/	API client, config, demo runner, models
src/app/features/	dashboard · request-demo
proxy.conf.json	
docs/	
screenshots/	the 8 images in §5
reports/	this report

The Angular app sits at the repository root as a sibling of the Maven module rather than nested inside it.

Redis API Rate Limiting and DDoS Protection POC — report generated 30 September 2026. All figures measured on the code in this repository.